Cross-Browser Automation with Selenium Using a Different Programming Language

Python

Is my chosen language aside from the lesson's JavaScript Selenium.

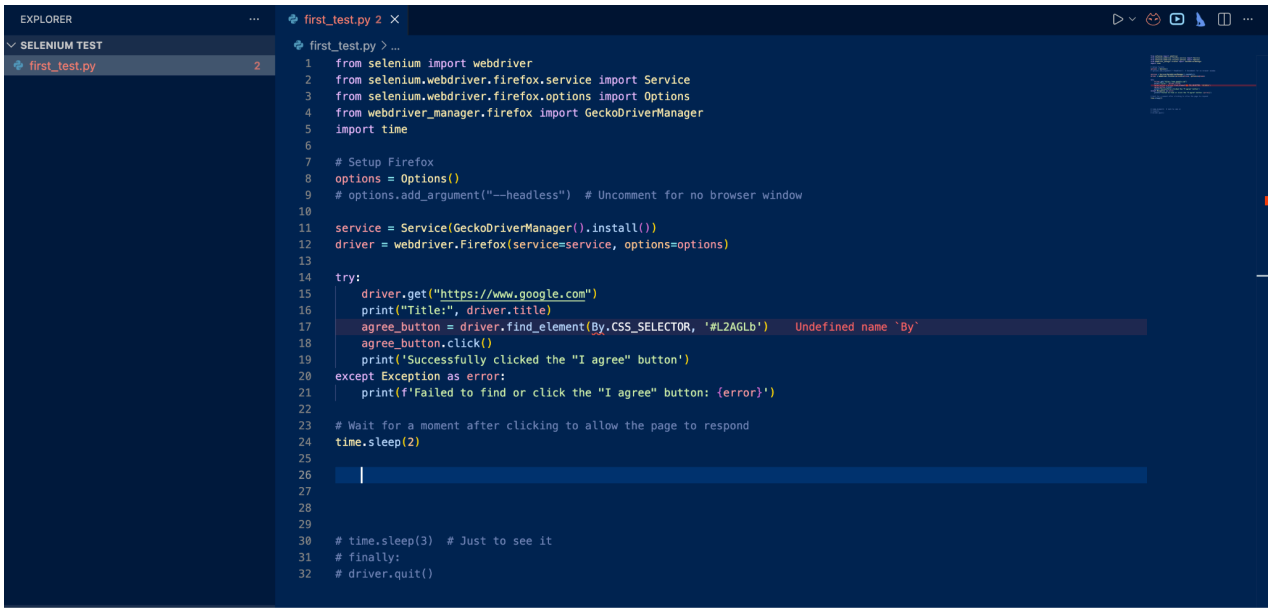
Firefox

Is my chosen browser aside from the lesson's JavaScript Selenium on Chrome.

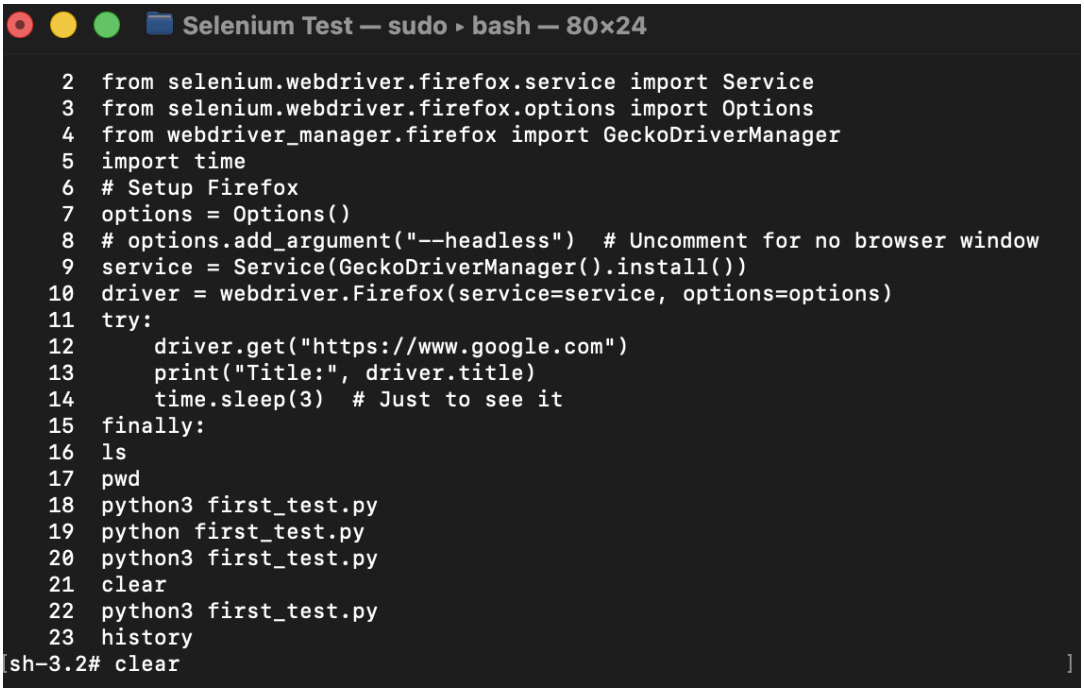
Preparations I did:

- Installed Python
- Installed Selenium Web Driver

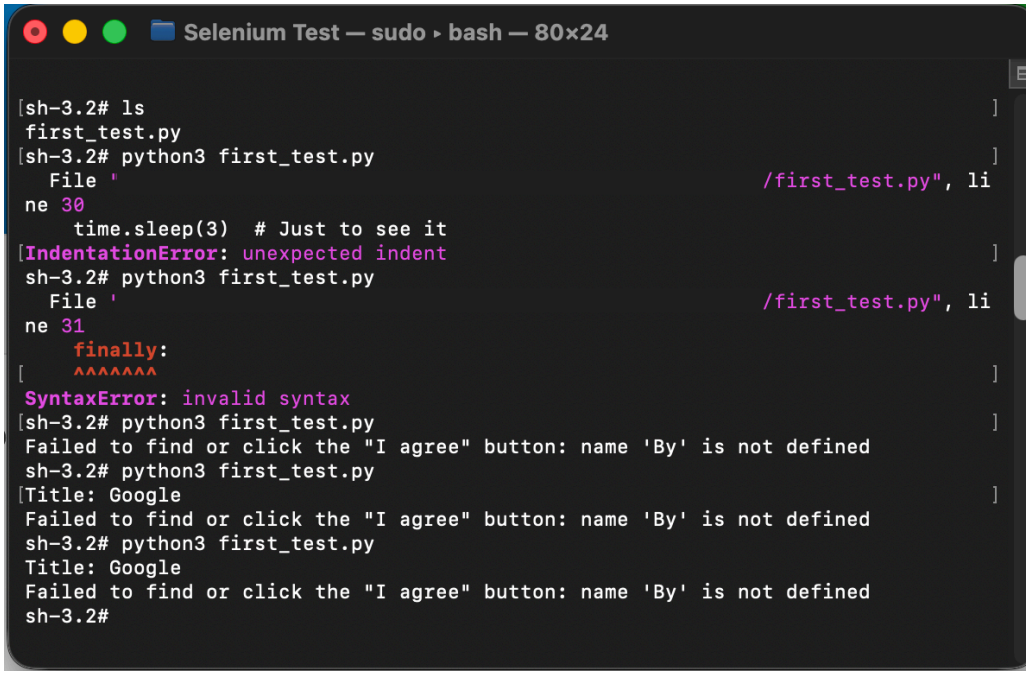
Ran the converted code on Python & Firefox environments



1. *first_test.py in Python version*

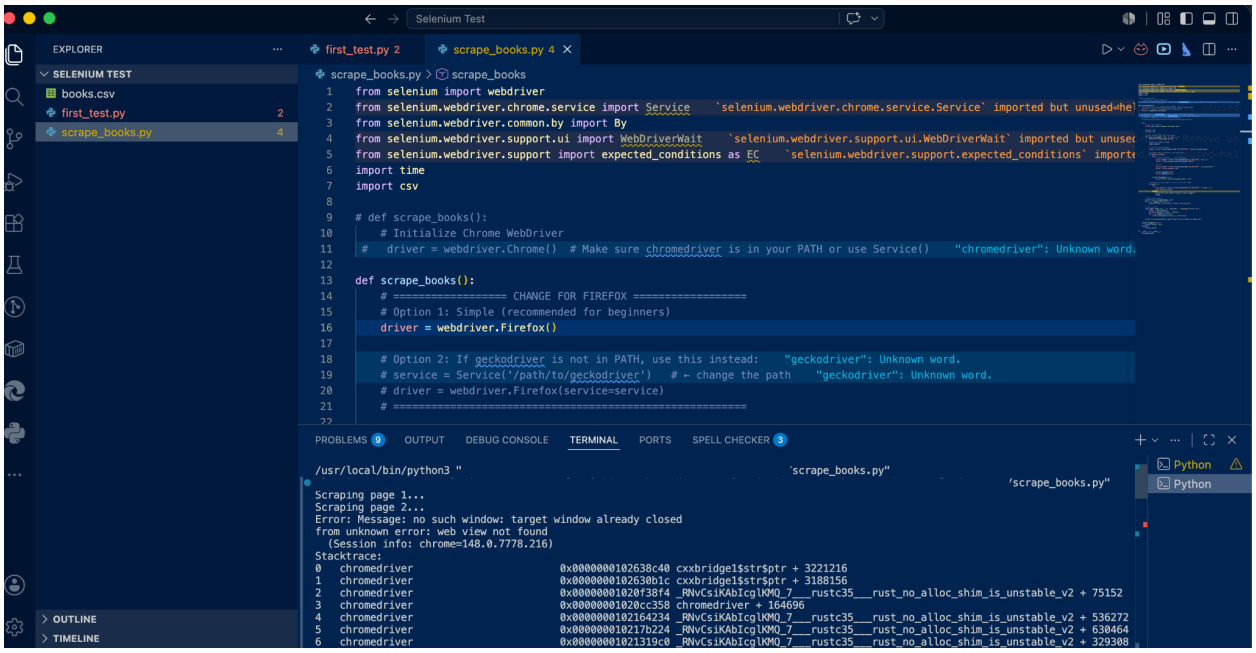


2. ***Proof of Running The Script:*** *first_test.py in Python version - First Run*

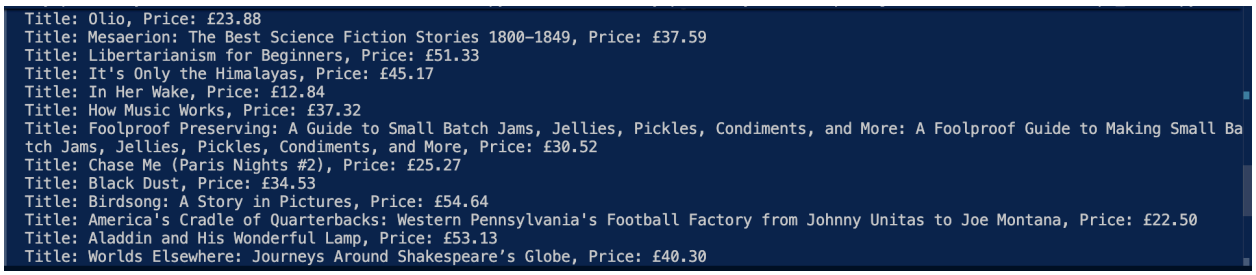


3. ***Proof of Running The Script:*** *first_test.py in Python version - Second Run with added lines and encountering errors*

Rewritten an Existing Script from JS to Python



4. ***Proof of Running The Script:*** *scrape_books.py as Python*



5. ***Output of the Run:*** *Scraped data as CSV*

Sharing My Findings

1. Syntax and Readability

- **JavaScript:** Relies heavily on curly braces `{ }`, semicolons `;`, and callbacks or Promises. It can look cluttered quickly.
- **Python:** Uses **indentation (whitespace)** to define code blocks. There are no semicolons. It reads almost like English, making your test scripts much cleaner and easier to maintain.

2. Asynchronous vs. Synchronous Execution

- **JavaScript:** Is inherently asynchronous. In Selenium (`wdio` or vanilla webdriver), you constantly have to manage `async/await` to ensure your test steps execute in the correct order. Missing an `await` will break your test.
- **Python:** Executes synchronously (line-by-line) by default. You don't need to sprinkle `async` and `await` everywhere; Python naturally waits for the previous Selenium command to finish before moving to the next line.

3. Driver Initialization & Setup

- **JavaScript:** Often requires a bit of boilerplate code or external test runners (like Mocha, Jasmine, or WebdriverIO) to manage the browser lifecycle efficiently.
- **Python:** Extremely straightforward. You can spin up a browser in just three lines of code using the native `unittest` or `pytest` frameworks.

Summary

- **Python** removes the mental tax of handling `async/await`.
- **Python** uses strict indentation instead of `{}` and `;`.
- **Python** has `pytest`, which is widely considered one of the most powerful and user-friendly test runners available.

Installation

- already installed Python on my device
- already installed Firefox browser on my device
- ran `pip install -U selenium` on my terminal - When I ran it, it downloaded the library alongside a built-in automation tool called Selenium Manager. When the script executes webdriver.Firefox(), this manager silently checks the computer's Firefox version, downloads the exact matching geckodriver from the internet in milliseconds, and saves it to a hidden cache folder for future use. This completely eliminates the old, manual setup process, allowing my Python code to initialize the browser instantly and run successfully out of the box.

Code Snippets

first_test.py

```
from selenium import webdriver

from selenium.webdriver.firefox.service import Service

from selenium.webdriver.firefox.options import Options

from webdriver_manager.firefox import GeckoDriverManager

import time



# Setup Firefox

options = Options()

# options.add_argument("--headless")  # Uncomment for no browser window



service = Service(GeckoDriverManager().install())

driver = webdriver.Firefox(service=service, options=options)



try:

   driver.get("https://www.google.com")

   print("Title:", driver.title)

   agree_button = driver.find_element(By.CSS_SELECTOR, '#L2AGLb')

   agree_button.click()

   print('Successfully clicked the "I agree" button')

except Exception as error:

   print(f'Failed to find or click the "I agree" button: {error}')



# Wait for a moment after clicking to allow the page to respond
```

```
time.sleep(2)









# time.sleep(3)  # Just to see it

# finally:

# driver.quit()
```

scrape_books.py

```
from selenium import webdriver

from selenium.webdriver.chrome.service import Service

from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

import time

import csv



# def scrape_books():

   # Initialize Chrome WebDriver

#   driver = webdriver.Chrome()  # Make sure chromedriver is in your PATH or use
Service()



def scrape_books():

   # ================== CHANGE FOR FIREFOX ==================
```

```python
# Option 1: Simple (recommended for beginners)
driver = webdriver.Firefox()

# Option 2: If geckodriver is not in PATH, use this instead:
# service = Service('/path/to/geckodriver')    # ← change the path
# driver = webdriver.Firefox(service=service)
# ============================================================

try:
    # Open the website
    driver.get('http://books.toscrape.com/')

    titles = []
    prices = []

    # Loop through the first two pages
    for page in range(1, 3):  # pages 1 to 2
        print(f"Scraping page {page}...")

        # Wait for page to load
        time.sleep(2)

        # Find all book elements
        books = driver.find_elements(By.CSS_SELECTOR, 'article.product_pod')

        # Extract title and price from each book
```

```
        for book in books:
            try:
                # Title from h3 > a title attribute
                title_element = book.find_element(By.CSS_SELECTOR, 'h3 > a')
                title = title_element.get_attribute('title')

                # Price
                price_element = book.find_element(By.CSS_SELECTOR, 'p.price_color')
                price = price_element.text

                titles.append(title)
                prices.append(price)

            except Exception as e:
                print(f"Error extracting book data: {e}")

        # Navigate to next page (if not on the last page)
        if page < 2:
            try:
                next_button = driver.find_element(By.CSS_SELECTOR, 'li.next > a')
                next_button.click()
            except:
                print("No next button found or end of pages.")
                break

    # Print the results
```

```
        print("\n=== Scraped Books ===")
        for i in range(len(titles)):
            print(f"Title: {titles[i]}, Price: {prices[i]}")

        # Save to CSV
        with open('books.csv', 'w', newline='', encoding='utf-8') as f:
            writer = csv.writer(f)
            writer.writerow(['Title', 'Price'])
            for i in range(len(titles)):
                writer.writerow([titles[i], prices[i]])

        print(f"\n✅ Successfully saved {len(titles)} books to books.csv")

    except Exception as e:
        print(f"Error: {e}")
    finally:
        driver.quit()

if __name__ == "__main__":
    scrape_books()
```

books.csv

Title,Price

A Light in the Attic,£51.77

Tipping the Velvet,£53.74

Soumission,£50.10

Sharp Objects,£47.82

Sapiens: A Brief History of Humankind,£54.23

The Requiem Red,£22.65

The Dirty Little Secrets of Getting Your Dream Job,£33.34

"The Coming Woman: A Novel Based on the Life of the Infamous Feminist, Victoria Woodhull",£17.93

The Boys in the Boat: Nine Americans and Their Epic Quest for Gold at the 1936 Berlin Olympics,£22.60

The Black Maria,£52.15

"Starving Hearts (Triangular Trade Trilogy, #1)",£13.99

Shakespeare's Sonnets,£20.66

Set Me Free,£17.46

Scott Pilgrim's Precious Little Life (Scott Pilgrim #1),£52.29

Rip it Up and Start Again,£35.02

"Our Band Could Be Your Life: Scenes from the American Indie Underground, 1981-1991",£57.25

Olio,£23.88

Mesaerion: The Best Science Fiction Stories 1800-1849,£37.59

Libertarianism for Beginners,£51.33

It's Only the Himalayas,£45.17

In Her Wake,£12.84

How Music Works,£37.32

"Foolproof Preserving: A Guide to Small Batch Jams, Jellies, Pickles, Condiments, and More: A Foolproof Guide to Making Small Batch Jams, Jellies, Pickles, Condiments, and More",£30.52

Chase Me (Paris Nights #2),£25.27

Black Dust,£34.53

Birdsong: A Story in Pictures,£54.64

America's Cradle of Quarterbacks: Western Pennsylvania's Football Factory from Johnny Unitas to Joe Montana,£22.50

Aladdin and His Wonderful Lamp,£53.13

Worlds Elsewhere: Journeys Around Shakespeare’s Globe,£40.30

Wall and Piece,£44.18

The Four Agreements: A Practical Guide to Personal Freedom,£17.66

The Five Love Languages: How to Express Heartfelt Commitment to Your Mate,£31.05

The Elephant Tree,£23.82

The Bear and the Piano,£36.89

Sophie's World,£15.94

Penny Maybe,£33.29

Maude (1883-1993):She Grew Up with the country,£18.02

"In a Dark, Dark Wood",£19.63

Behind Closed Doors,£52.22

You can't bury them all: Poems,£33.63